

目录

项目概览	3
架构设计	3
三层架构	3
数据流	4
双执行模式	4
核心包深度分析	5
apps/daemon - 后端引擎	5
Agent 适配器模式	5
插件引擎	5
Critique Theater - AI 设计评审	6
媒体生成	6
apps/web - 前端应用	6
apps/desktop - Electron 桌面壳	7
packages/contracts - 共享契约层	7
packages/plugin-runtime - 纯插件引擎	7
包依赖关系	7
关键架构模式	8
提示词分层组合	8
SSE 流式通信协议	9
Artifact 流式渲染	9
插件即文件系统	10
设计系统即 Markdown	10
Monorepo 目录结构	10
技术栈全景	11

核心能力总结	12
Agent 无关的设计引擎	12
实时流式设计预览	13
分层提示词工程	13
插件化设计流水线	13
AI 驱动的多专家评审	13
本地优先与安全设计	13
重要架构决策	14
生态体系	15
测试与质量保障	16
总结与评价	16

项目概览

Open Design 是一个开源的本地优先（local-first）设计产品，定位为"Agent 时代的 Figma 替代品"和"开源版 Claude Design"。该项目由 nexu-io 团队开发，版本号为 0.10.0，采用 Apache-2.0 许可证，已有 88+ 位贡献者参与。其核心理念是：不构建独立的云端设计工具，而是将用户已有的编码 Agent CLI（如 Claude Code、Codex、Cursor、Copilot 等 21 种工具）转化为设计引擎，通过读取 SKILL.md（设计技能）和 DESIGN.md（设计系统/品牌令牌）文件，将 HTML 产物流式渲染到沙箱预览窗口中。

与传统的 Figma 等设计工具不同，Open Design 不依赖云端服务，所有功能均可在本地运行。它采用 pnpm monorepo 架构，基于 Node 24 和全 TypeScript 技术栈，通过 Express 后端启动 Agent CLI 进程，将 Agent 的标准输出流解析为 SSE 事件，并实时推送到 Next.js 前端进行渲染。这种架构使得 Open Design 能够兼容几乎所有主流的 AI 编码工具，而非绑定特定的模型或平台。

属性	值
项目名称	Open Design
版本	0.10.0
许可证	Apache-2.0
架构模式	pnpm Monorepo
运行时	Node 24 + TypeScript
贡献者	88+
支持 Agent 数	21 种 CLI
官方插件数	261 个
品牌设计系统	150 套

表 1: 项目基本信息

架构设计

三层架构

Open Design 采用经典的三层架构设计，将系统分为前端展示层、后端守护进程层和 Agent 执行层。这种分层设计实现了关注点的清晰分离：前端专注于用户交互和可视化渲染，守护进程负责 Agent 管理、流式通信和插件编排，而 Agent 层则专注于实际的代码生成和设计推理。三层之间通过 HTTP API 和 SSE 流式协议进行通信，保证了层间的松耦合。

层级	技术栈	职责
前端展示层	Next.js 16 + React 18 + Tailwind CSS 4	用户界面、聊天交互、沙箱预览、插件管理
守护进程层	Express 5 + SQLite (better-sqlite3)	Agent 生命周期管理、SSE 流式转发、插件引擎、MCP 服务器
Agent 执行层	21 种 AI Agent CLI	读取 SKILL.md/DESIGN.md、生成 HTML 产物、流式输出

表 2：三层架构职责划分

数据流

Open Design 的数据流遵循"用户输入 - 后端编排 - Agent 执行 - 实时渲染"的完整闭环。用户在前端 Studio 界面输入设计需求后，Web 应用将请求发送到守护进程的 /api/chat 端点。守护进程随后进行系统提示词的分层组装：从基础设计器提示词开始，逐层叠加发现层、设计系统、技能、插件块和记忆上下文，形成完整的系统提示词。组装完成后，守护进程根据检测到的 Agent 类型，在项目工作目录中启动对应的 Agent CLI 进程，并通过标准输出流实时捕获 Agent 的输出。

Agent 的输出经过守护进程的 SSE 事件解析器处理后，被规范化为统一的事件流（包括文本增量、工具调用、artifact 块等），然后通过 SSE 协议转发到前端。前端的 artifact 解析器实时从 Agent 输出中提取 HTML 内容，并将其渲染到沙箱化的 srcdoc iframe 中，实现即时的设计预览。最终，所有产物文件被持久化到 .od/projects/ 目录下的文件系统中，便于后续的版本管理和迭代。

双执行模式

Open Design 支持两种执行模式，以适应不同用户的使用场景和技术偏好。第一种是本地 CLI 模式（默认模式），守护进程直接在用户的本地环境中检测并启动已安装的 Agent CLI 二进制文件，通过标准输入输出与 Agent 交互。这种模式的优点是完全本地化、无需 API Key、数据不离开本地，适合对隐私和安全性有较高要求的用户。

第二种是 API 模式（BYOK, Bring Your Own Key），守护进程作为代理服务器，将请求转发到 OpenAI、Anthropic、Azure、Google 或 Ollama 等兼容 OpenAI API 格式的端点。守护进程在转发前会进行 SSRF 防护检查，确保请求的安全性。这种模式适合已经在使用云端 API 服务的用户，或者需要在多个模型之间切换的场景。两种模式共享同一套 SSE 事件规范，因此前端无需感知底层的执行模式差异。

模式	工作方式	适用场景	安全特性
本地 CLI	守护进程启动 Agent 二进制文件，流式读取 stdout	本地优先、无需 API Key	数据不离开本地
API (BYOK)	代理到 OpenAI/Anthropic/Azure/Google/Ollama	云端 API 用户、多模型切换	SSRF 防护、端点校验

表 3：双执行模式对比

核心包深度分析

apps/daemon - 后端引擎

daemon 是 Open Design 的心脏，一个基于 Express 5 的 HTTP 服务器，使用 better-sqlite3 作为本地数据库。它承担了系统中最核心的职责：Agent 生命周期管理、SSE 流式通信、插件系统编排、设计系统管理、媒体生成、MCP 服务器等。daemon 的设计哲学是"Agent 无关"—它不自带 AI 模型，而是编排用户已安装的任何 CLI Agent，将不同 Agent 的输出格式统一为标准 SSE 事件流，使上层前端无需关心底层 Agent 的差异。

Agent 适配器模式

每种支持的 Agent CLI 都有一个专属适配器模块，位于 apps/daemon/src/runtimes/default/ 目录下。适配器定义了：二进制文件解析路径、参数构建逻辑、流式输出格式、模型解析策略和认证诊断方法。添加一个新的 Agent 支持只需创建一个新文件并在注册表中注册即可，完全遵循开闭原则。当前已支持的 21 种 Agent 包括：Claude Code、Codex、Cursor Agent、GitHub Copilot、Gemini、Aider、AMR 等，覆盖了主流的 AI 编码工具生态。

插件引擎

daemon 内置了完整的插件生命周期管理引擎，涵盖安装、应用、脚手架生成、验证、发布和市场化等环节。插件通过声明 od.pipeline.stages 数组来定义流水线阶段，每个阶段运行

命名的"原子" (atoms)，如 discovery-question-form、direction-picker、critique-theater 等。阶段可以设置 until 条件实现循环执行，直到满足退出标准。第一方原子实现位于 apps/daemon/src/plugins/atoms/ 目录下，包括 Figma 提取、代码导入、差异审查、设计交接等功能。插件还声明 od.capabilities 数组来控制权限，支持 prompt:inject、fs:read、fs:write、mcp、bash、network、connector 等能力声明。受限安装默认只获得 prompt:inject 权限，确保了安全性。

Critique Theater - AI 设计评审

这是一个创新的 AI 设计评审系统，模拟 5 位评审专家（设计师、评论家、品牌专家、无障碍专家、文案专家）对生成的设计产物进行多轮评分和反馈。系统基于 Reducer 状态机实现，支持配置化的评分阈值和最大轮次。评审过程通过 SSE 协议实时推送到前端，用户可以观看评审直播或回放历史评审。系统还集成了 Prometheus 指标，并采用滚动式灰度发布策略（Rollout Ratchet），通过 16 个阶段的渐进式启用控制功能的发布节奏。

媒体生成

daemon 支持多种媒体生成能力，覆盖图像、视频和音频三大类型。图像生成支持 gpt-image-2 和 Leonardo；视频生成支持 Seedance、Veo、Kling 和 HyperFrames；音频生成支持 ElevenLabs 和 SenseAudio。这些媒体生成能力通过统一的 media-adapters 接口集成，使得在设计中嵌入多媒体内容变得便捷。

apps/web - 前端应用

前端应用基于 Next.js 16（App Router）和 React 18 构建，使用 Tailwind CSS 4 进行样式管理，Turbopack 作为开发构建工具。应用的核心是 App.tsx 组件，它编排了模式/技能/设计系统选择器、聊天编辑器和预览工作区。前端的关键技术亮点包括：流式 artifact 解析器（从 Agent 输出中实时提取 HTML）、SSE 传输层（与守护进程通信）、BYOK 代理层（直连 AI 端点）、国际化系统（支持 18+ 语言环境）以及可观测性系统（白屏检测、启动计时、卡顿检测）。

前端的 artifact 解析器是一个精巧的流式 HTML 提取器，能够从 Agent 的 SSE 输出中实时识别 标签，并提取其中的 HTML 内容渲染到沙箱化的 srcdoc iframe 中。该解析器能够处理嵌套标签、代码围栏和增量流式输出，实现了"边生成边预览"的即时设计体验。此外，前端还包含完整的 Critique Theater UI，基于 useCritiqueStream 和 useCritiqueReplay 两个自定义 Hook 实现了评审直播和回放功能。

apps/desktop - Electron 桌面壳

桌面应用是一个轻量级的 Electron 41 壳层，负责在 BrowserWindow 中加载 Web 应用，并通过注入的 `__od__` 全局对象提供原生能力桥接。桥接协议定义在 `@open-design/host` 包中，包括文件系统访问（打开外部文件/文件夹）、PDF 导出、截图、自动更新、桌面认证门控等功能。桌面端还通过 Sidecar IPC 协议（Unix Socket / Windows Named Pipe）与守护进程通信，支持状态查询、JavaScript 评估、截图、PDF 导出、自动更新等操作。为了安全，文件夹导入使用 HMAC 签名令牌进行认证，防止渲染进程访问任意路径。

packages/contracts - 共享契约层

contracts 是整个系统的单一事实来源（Single Source of Truth），定义了 Web 与守护进程之间的所有接口契约。该包采用纯 TypeScript + Zod 实现，零 Node.js 依赖（仅依赖 zod），因此可以同时守护进程（Node 环境）和 Web（浏览器环境）中使用。contracts 包含：API 类型定义（ChatRequest、ChatMessage、PersistedAgentEvent 等）、插件清单的 Zod 模式（open-design.json 的完整校验规则）、SSE 事件协议类型、提示词组合逻辑（composeSystemPrompt 函数）、评审配置模式、分析事件模式以及设计系统令牌模式。其中，composeSystemPrompt 函数是系统提示词的核心组装器，按照 14 层优先级叠加各层提示词，确保了提示词的一致性和可预测性。

packages/plugin-runtime - 纯插件引擎

plugin-runtime 是一个纯粹的插件运行时引擎，没有任何 node:fs 导入，所有文件加载由守护进程或 Web 端注入。该包提供：Frontmatter 解析器、清单解析器、市场适配器、Agent-Skill 适配器、Claude-Plugin 适配器、插件配置合并、摘要计算、清单验证、引用解析和流水线降级处理。这种设计使得插件系统可以在任何 JavaScript 环境中运行，包括浏览器端的插件预览和验证。

包依赖关系

Open Design 的包依赖关系遵循"契约层为锚点"的设计原则。contracts 包作为中心共享库，被 daemon 和 web 两个核心应用同时依赖，确保了前后端接口的一致性。sidecar-protocol 定义了进程间通信协议，被 sidecar 运行时实现，然后被 daemon、web 和 desktop 共同使用。launcher-protocol 基于 sidecar-protocol 扩展，用于自动更新协议，被 desktop 和 packaged 使用。host 包定义了桌面桥接协议，仅被 web 和 desktop 使用。这种分层依赖关

系确保了每个包的职责单一和可替换性。

包名	被依赖方	核心职责
contracts	daemon, web	API 类型、Zod 模式、SSE 协议、提示词组合
plugin-runtime	daemon	插件解析、验证、适配（纯逻辑，无 I/O）
sidecar-proto	sidecar, daemon, web, desktop	进程间通信协议定义
sidecar	daemon, web, desktop	通用进程编排原语（端口分配、IPC、路径解析）
launcher-proto	desktop, packaged	自动更新协议定义
host	web, desktop	桌面桥接协议（原生能力暴露）
components	web	共享 React UI 组件库
platform	daemon, download	通用进程/平台原语
diagnostics	daemon, desktop	日志收集、脱敏、打包
registry-protocol	daemon	插件注册中心后端协议
agui-adapter	daemon	AG-UI (CopilotKit) 双向事件适配

表 4：核心包依赖关系

关键架构模式

提示词分层组合

系统提示词的组装是 Open Design 最核心的架构创新之一。composeSystemPrompt 函数按照严格的 14 层优先级叠加各层提示词，从最高优先级的 API 模式覆盖层，到最低优先级的媒体生成契约层。中间层依次为：聊天模式覆盖、发现与哲学层、基础设计器章程、个人记忆、用户级指令、项目级指令、活跃设计系统（DESIGN.md）、活跃技能（SKILL.md）、插件块、活跃流水线阶段、项目元数据、演示文稿框架。这种分层设计确保了每一层都可以独立修改而不影响其他层，同时保证了优先级的确定性。例如，用户级指令可以覆盖基础设计器的默认行为，而设计系统令牌又可以被技能定义进一步限定。

优先级	层级名称	说明
-----	------	----

1（最高）	API 模式覆盖	BYOK 模式下的系统提示词覆盖
2	聊天模式覆盖	不同聊天模式（设计/对话）的提示词差异
3	发现与哲学层	Agent 能力发现和设计哲学声明
4	基础设计器章程	核心设计原则和约束条件
5	个人记忆	从历史对话中自动提取的上下文
6	用户级指令	用户全局偏好和指令
7	项目级指令	当前项目的特定指令
8	设计系统 (DESIGN.md)	品牌令牌、组件规范、样式约束
9	技能 (SKILL.md)	特定设计技能的指令和示例
10	插件块	活跃插件的上下文注入
11	活跃流水线阶段	当前流水线阶段的指令
12	项目元数据	项目类型、平台、保真度等信息
13	演示文稿框架	演示文稿模式的固定结构
14（最低）	媒体生成契约	图像/视频/音频生成的约束条件

表 5：提示词分层组合优先级

SSE 流式通信协议

守护进程与前端的通信采用结构化的 SSE（Server-Sent Events）协议，确保了实时性和可靠性。协议定义了以下事件类型：start 事件携带运行元数据（Agent 类型、模型、工作目录）；agent 事件包含状态更新、文本增量（text_delta）、思维增量（thinking_delta）、工具调用（tool_use）、工具结果（tool_result）和使用统计（usage）；stdout/stderr 事件传递原始 CLI 输出；end 事件标记流结束，包含退出码、状态和可恢复标志。这种统一的事件规范使得前端能够以相同的方式处理来自不同 Agent 的输出，实现了 Agent 的可替换性。

Artifact 流式渲染

前端的 标签解析器是用户体验的核心技术。该解析器从 Agent 的 SSE 输出流中实时识别标签，提取其中的 HTML 内容，并将其渲染到沙箱化的 srcdoc iframe 中。解析器需要处理嵌套标签、代码围栏和增量流式输出等复杂场景。当 Agent 正在生成代码时，用户就可以看到

设计的实时预览，这种"边生成边预览"的体验大幅缩短了从想法到视觉呈现的反馈循环。沙箱化的 `iframe` 设计确保了生成的 HTML 不会影响主应用的安全性。

插件即文件系统

Open Design 的插件本质上就是包含 `SKILL.md` 文件和可选 `open-design.json` 清单文件的文件夹。这种设计无需构建步骤，无需运行时沙箱——Agent 直接将插件内容作为上下文读取。插件的 `open-design.json` 清单支持声明：插件类型（`kind`）、任务类型（`taskKind`）、运行模式（`mode`）、流水线阶段（`pipeline.stages`）、GenUI 界面（`genui`）、输入定义（`inputs`）和能力声明（`capabilities`）。当前项目包含 261 个官方插件，覆盖了场景（`scenarios`）、原子（`atoms`）、设计系统（`design-systems`）和示例（`examples`）四大类别。社区插件目录下还有 `hallmark`、`import-smoke-test` 和 `registry-starter` 等项目。

设计系统即 Markdown

项目收录了 150 套品牌设计系统，每套系统由一个 `DESIGN.md` 文件定义，遵循 9 段式结构规范。这种选择颇具深意：不使用 JSON 或 CSS 变量，而是使用 Markdown 格式，因为 AI Agent 对 Markdown 的理解和生成能力远强于其他格式。设计系统包含品牌令牌（颜色、字体、间距）、组件规范（按钮、卡片、表单等）和样式约束，Agent 在生成设计时会将 `DESIGN.md` 中的品牌规范作为权威约束，确保产出与品牌调性一致。知名品牌系统包括 Stripe、Apple、Linear、Vercel、GitHub 等，用户也可以导入自定义的设计系统。

Monorepo 目录结构

Open Design 采用 `pnpm workspace` 管理的 `monorepo` 结构，工作区配置覆盖 `packages/*`、`apps/*`、`tools/*` 和 `e2e` 四个区域。这种结构使得各应用和包可以独立开发、测试和发布，同时共享统一的构建工具链和版本管理。以下是关键目录的功能说明：

目录	类型	说明
<code>apps/daemon</code>	应用	Node/Express 后端 - 启动 Agent、提供 API、SQLite 数据库
<code>apps/web</code>	应用	Next.js 16 App Router + React 18 客户端
<code>apps/desktop</code>	应用	Electron 桌面壳（macOS + Windows + Linux）
<code>apps/packaged</code>	应用	桌面打包编排器

apps/landing-page	应用	Astro 营销网站
apps/telemetry-worker	应用	Cloudflare Workers 遥测接收器
packages/contracts	包	共享纯 TS 契约（类型、Zod 模式、提示词）
packages/plugin-runtime	包	插件清单解析器、适配器、验证器
packages/host	包	渲染器宿主桥接协议（Electron 与 Web 通信）
packages/sidecar-protocol	包	Sidecar IPC 协议契约
packages/sidecar	包	通用 Sidecar 运行时原语
packages/agui-adapter	包	AG-UI (CopilotKit) 双向事件适配
packages/components	包	共享 React UI 原语组件
plugins/_official	生态	261 个官方插件
design-systems	生态	150 套 DESIGN.md 品牌系统
design-templates	生态	技能模板资产
skills	生态	SKILL.md 技能定义
craft	生态	设计工艺指南（排版、色彩、动画、无障碍）
e2e	测试	Playwright + Vitest 端到端测试套件

表 6: Monorepo 关键目录结构

技术栈全景

Open Design 的技术栈选择体现了"成熟稳定 + 前沿探索"的平衡策略。后端采用 Express 5 和 SQLite，保证了轻量化和本地优先的特性；前端使用 Next.js 16 和 React 18，紧跟 React 生态的最新发展；桌面端基于 Electron 41，提供了跨平台的原生体验。全项目使用 TypeScript 5.9+ 作为统一语言，配合 esbuild 进行快速打包，Vitest 进行单元测试，Playwright 进行端到端测试。

类别	技术	用途
后端框架	Express 5	HTTP 服务器 + API 路由
数据库	better-sqlite3	本地 SQLite 数据库
MCP	@modelcontextprotocol/sdk	MCP 服务器实现
类型校验	Zod	运行时类型验证
前端框架	Next.js 16 (App Router)	React 框架
UI 库	React 18 + Tailwind CSS 4	用户界面
富文本编辑	Lexical	聊天编辑器
终端	@xterm/xterm	终端 UI 组件
动画	Motion (Framer Motion)	界面动画
桌面端	Electron 41	原生桌面壳
构建工具	esbuild 0.28 + Turbopack	快速打包
测试	Vitest 4 + Playwright	单元测试 + E2E 测试
监控	PostHog + Prometheus	分析 + 指标
静态站点	Astro 6	营销落地页
AI 编码	z-ai-web-dev-sdk	AI 模型调用

表 7：技术栈全景

核心能力总结

Agent 无关的设计引擎

Open Design 最核心的能力是作为"Agent 无关"的设计引擎。它不绑定任何特定的 AI 模型或平台，而是通过 21 种 Agent 适配器兼容几乎所有主流 AI 编码工具。用户可以在 Claude Code、Codex、Cursor、Copilot 等工具之间自由切换，系统会自动检测并适配对应的 Agent。这种设计极大地降低了用户的切换成本，也使得 Open Design 能够随着 AI 工具生态的演进而自然扩展，而不需要进行大规模的架构改造。添加新 Agent 只需编写一个适配器文件并注册，完全遵循开闭原则。

实时流式设计预览

通过 artifact 流式解析器和沙箱化 iframe 渲染, Open Design 实现了"边生成边预览"的即时设计体验。当 Agent 正在输出 HTML 代码时, 前端就能实时捕获并渲染预览, 用户无需等待完整的代码生成完毕。这种实时反馈循环极大地提升了设计迭代效率, 将传统"生成-查看-修改"的串行流程转变为"边看边改"的并行流程。SSE 协议确保了事件传递的低延迟和可靠性, 即使网络不稳定也能保持连接。

分层提示词工程

14 层提示词组合系统是 Open Design 在 AI 工程方面的核心创新。通过将提示词拆分为独立的层, 每层可以独立开发、测试和迭代, 而不影响其他层的稳定性。优先级机制确保了冲突时的确定性解决, 而分层叠加的方式使得复杂的设计约束(品牌规范、技能定义、项目元数据等)能够被有机地融合到一个系统提示词中。这种模式将"提示词工程"从简单的文本编写提升为"提示词架构", 为复杂 AI 应用的提示词管理提供了可复用的方法论。

插件化设计流水线

插件系统采用了"声明式流水线 + 原子化执行"的模式。插件通过 open-design.json 声明流水线阶段, 每个阶段运行命名的原子操作, 阶段可以设置循环条件实现迭代优化。这种设计使得复杂的设计流程(如发现-方向-评审-交付)可以被编排为自动化的流水线, 而不是依赖用户手动触发每个步骤。261 个官方插件和活跃的社区生态为用户提供了丰富的开箱即用功能, 同时声明式的插件格式使得创建新插件的门槛极低——只需一个 SKILL.md 文件即可。

AI 驱动的多专家评审

Critique Theater 是 Open Design 在质量保障方面的创新实践。通过模拟 5 位不同视角的评审专家, 系统对生成的设计进行多维度评估: 设计美学、品牌一致性、无障碍合规和文案质量。多轮迭代机制允许系统根据评审反馈自动改进设计, 直到达到配置的评分阈值。这种"AI 评审 AI"的模式在 AI 应用中属于前沿实践, 它将传统的设计评审流程自动化, 同时保持了多视角评审的深度和广度。滚动式灰度发布策略确保了新功能的可控上线。

本地优先与安全设计

Open Design 坚持本地优先的架构原则：所有核心功能在 localhost 上运行，数据存储在本地的 SQLite 数据库中，守护进程默认绑定 127.0.0.1。BYOK 代理模式进行 SSRF 防护检查，桌面端的文件夹导入使用 HMAC 签名令牌认证，防止渲染进程访问任意路径。插件的能力声明系统（capabilities）通过白名单机制限制了插件的操作权限，受限安装默认只获得 prompt:inject 权限。这种多层安全设计在保持功能灵活性的同时，有效降低了供应链攻击和权限滥用等安全风险。

重要架构决策

Open Design 的架构中蕴含了多项深思熟虑的设计决策，这些决策不仅影响了代码结构，更塑造了产品的核心特质。以下是十个最值得关注的架构决策及其背后的权衡考量：

决策 1：本地优先，无需云端

所有功能在 localhost 运行，SQLite 存储，守护进程绑定 127.0.0.1。BYOK 代理有 SSRF 防护。

权衡考量：保证数据隐私和离线可用性，降低服务器成本。

决策 2：Agent 无关设计

不自带 AI 模型，编排用户已安装的 CLI Agent，统一输出格式。

权衡考量：避免模型锁定，兼容生态演进，降低用户迁移成本。

决策 3：插件即文件系统

插件是包含 SKILL.md + open-design.json 的文件夹，无需构建步骤。

权衡考量：极低门槛创建插件，Agent 天然理解 Markdown，无需沙箱。

决策 4：设计系统即 Markdown

150 套品牌系统采用 DESIGN.md 格式而非 JSON/CSS 变量。

权衡考量：AI Agent 对 Markdown 理解能力最强，生成质量最高。

决策 5：契约层为架构缝合点

contracts 包是 daemon/web 的唯一共享边界，纯 TS + Zod。

权衡考量：保证前后端接口一致，双端可消费，零 Node 依赖。

决策 6：Sidecar IPC 进程编排

daemon/web/desktop 通过 JSON-over-Unix-Socket IPC 通信。

权衡考量：进程隔离，独立崩溃恢复，跨平台统一通信。

决策 7：评审灰度发布

Critique Theater 采用 16 阶段滚动式灰度发布。

权衡考量：降低新功能上线风险，渐进式验证稳定性。

决策 8：能力门控插件

插件声明 capabilities 数组控制权限，受限安装默认仅 prompt:inject。

权衡考量：防止恶意插件越权操作，供应链安全保障。

决策 9：HMAC 文件夹导入

桌面端使用 HMAC 签名令牌认证文件夹导入。

权衡考量：防止渲染进程访问任意路径的安全漏洞。

决策 10：提示词级国际化

系统提示词包含 locale 特定指令，Agent 生成本地化 UI 文本。

权衡考量：从源头实现国际化，而非仅翻译界面字符串。

生态体系

Open Design 的生态体系是其核心竞争力之一。项目不仅提供工具本身，还构建了完整的插件市场、设计系统库和技能模板库，形成了一个“工具 + 内容 + 社区”的三位一体生态。这种生态化策略降低了用户的冷启动成本，同时通过社区贡献实现了功能的持续扩展。

生态组件	规模	说明
官方插件	261 个	覆盖场景、原子、设计系统、示例四大类别

品牌设计系统	150 套	Stripe、Apple、Linear、Vercel、GitHub 等知名品牌
技能模板	多类型	web-prototype、html-ppt、hyperframes 等
提示词模板	93 图 + 39 视频	图像和视频生成提示词库
设计工艺指南	4 领域	排版、色彩、动画、无障碍
社区插件	持续增长	hallmark、import-smoke-test、registry-starter
插件注册中心	完整	插件发布、发现、安装基础设施

表 8：生态体系概览

测试与质量保障

Open Design 建立了多层次的质量保障体系，从单元测试到端到端测试，从类型检查到插件验证，覆盖了代码质量、功能正确性和产品一致性三个维度。

- **单元测试：**使用 Vitest 覆盖所有包，contracts 包有 20+ 测试文件，daemon 包覆盖插件系统、评审引擎和 Agent 管理，tools/pack 有 30+ 测试文件。
- **端到端测试：**Playwright 测试框架位于 e2e/ 目录，包含 Mock OpenAI 服务器、HTTP 辅助工具、冒烟测试套件、平台专属规格（macOS/Windows/Linux）和功能覆盖率测试。
- **插件验证：**提供 od plugin validate 命令和 pnpm guard 脚本，执行样式策略检查、产品中立性检查和 Fork PR 工作流验证。
- **类型检查：**pnpm typecheck 命令在所有工作区包上运行 TypeScript 类型检查，确保类型安全。

总结与评价

Open Design 是一个架构设计精良、工程实践优秀的开源项目。它成功地将"AI Agent 编排"这一新兴需求转化为可扩展的软件架构，通过 Agent 适配器模式、分层提示词组合、声明式插件流水线等创新设计，实现了 Agent 无关、本地优先、插件化扩展的产品特性。

项目的技术亮点在于：一是"契约层为锚点"的架构缝合策略，通过 contracts 包确保前后端接口的一致性；二是"文件即插件"的极简设计，将插件门槛降到最低，同时保持了强大的流水线编排能力；三是"多专家 AI 评审"的创新实践，将传统的质量保障流程与 AI 能力深度结合。

从架构演进角度看，Open Design 的"Agent 无关"策略具有显著的前瞻性。随着 AI 编码工具的快速迭代，新的 Agent 不断涌现，Open Design 只需添加适配器文件即可兼容，无需调整核心架构。这种可扩展性使得项目能够持续跟进行业发展，保持长期的技术生命力。同时，本地优先的架构选择也契合了当前用户对数据隐私和工具可控性日益增长的需求。

总的来说，Open Design 不仅是一个实用的设计工具，更是 AI Agent 编排架构的优秀实践案例。其分层解耦、插件化扩展、流式渲染等架构模式，对于构建类似的 AI 原生应用具有很高的参考价值。项目当前版本为 0.10.0，仍在快速迭代中，值得关注其后续发展。